

Subagents

Module 7 · delegate work to a second Claude with its own context

Lab: L05 Subagents

OBJECTIVES

What you will be able to do

- Say when one agent hits a ceiling and a subagent is the fix
- Write a subagent file with the right tools, model and prompt
- Predict what a subagent can and cannot see
- Use the built-in subagents and forks
- Write a delegation with a goal, inputs, an output and a stop condition
- Choose foreground or background, and watch running work
- Choose between a rule, a skill, a subagent and a hook

7.1 · THE CEILING

Where one agent hits a ceiling

One agent in one conversation works well until the task gets big. Four limits show up.

Context fills

Test logs, search results and file dumps stay in the window and push out what matters (M1, M3)

Work is serial

Two independent reviews take twice as long

One permission set

The agent that writes the code can also approve it

Marks its own homework

The agent that wrote the code reviews it with the same blind spots

What a subagent is, and when to use one

A subagent is a second Claude with its own context window, system prompt, tools and permissions. You hand it a task; only its final summary comes back.

Use a subagent for

- Noisy side work (tests, logs, searches) where only the result matters
- A tool restriction: a reviewer that cannot edit
- Self-contained work that can run in parallel
- A second opinion from something that did not write the code

vs

Stay in the main conversation for

- Back-and-forth where each step depends on the last
- Quick, targeted edits
- Anything where speed matters: a fresh subagent must gather context first
- A question about what is already in your chat: use /btw

The subagent file

A Markdown file with YAML frontmatter. The body is its system prompt. Only name and description are required.

.CLAUDE/AGENTS/TODO-REVIEWER.MD

```
---
name: todo-reviewer
description: Reviews the branch diff against the
  to-do rules R1-R8 and for missing tests.
  Use after code changes, before a pull request.
tools: Read, Grep, Glob, Bash
model: sonnet
maxTurns: 15
---
You are a code reviewer. You never change files.
Review git diff main...HEAD. Return VERDICT
(APPROVE or CHANGES), then numbered findings.
```



Frontmatter

Who it is, when to use it, what it may do.



Body

Its system prompt. It replaces the normal Claude Code prompt.

Where agents live: the higher row wins

#	LOCATION	SHARED WITH
1	Managed settings	The whole organisation
2	`claude --agents '<json>'`	This session only
3	`.claude/agents/`	The team, through git
4	`~/.claude/agents/`	You, in all projects
5	A plugin's agents/ folder (M13)	Whoever installs the plugin

Folders are scanned recursively. Only name identifies an agent, so keep names unique. /doctor reports duplicates.

The fields you will use, and which model runs

FIELD (CAMELCASE)	PURPOSE
`name`	Unique id. No colon
`description`	When Claude should delegate to it
`tools`	Allow-list. Omit it: inherits every tool
`disallowedTools`	Deny-list, applied first
`model`	sonnet, opus, haiku, fable, an id, or inherit
`maxTurns`	Turn cap; output marked partial at the cap
`skills`	Skills preloaded in full (M6)
`background`	true: always run in the background

Which model runs: first match wins

1. The model Claude passes for that call
2. The file's model field
3. CLAUDE_CODE_SUBAGENT_MODEL
4. The session's model

A cheap haiku for a read-only searcher is a common saving.

Things that bite

Silent ignores

Unknown or misspelled fields are ignored. A file with no name or description is skipped. claude --debug shows why.

Reloading

Edits are picked up within seconds. Created the agents folder after the session started? Restart.

/agents

Now only prints a reminder: ask Claude to create or edit an agent, or edit the file yourself.

EASIEST WAY TO CREATE ONE

```
Create a subagent in .claude/agents/finder.md that finds where something is defined
or used. Tools Read, Grep, Glob only. Model haiku. Return a table: file, line, note.
```

What a subagent sees

It gets

- Its own system prompt (the file body)
- The delegation message
- CLAUDE.md files and git status (not Explore or Plan)
- Skills named in its skills: field

It does not get

- Your conversation history
- Files you already read
- Skills you already invoked
- Auto memory

Not in the message, a file or CLAUDE.md? It does not know it. Most "it got it wrong" bugs are missing context.

Tool output stays inside: 10,000 lines of test output cost its window, not yours.

Built-in subagents

AGENT	USE	NOTES
`Explore`	Fast read-only search: quick, medium or very thorough	No edits. Skips CLAUDE.md and git status
`Plan`	Read-only research in plan mode (M4)	Skips CLAUDE.md. Cannot be resumed
`general-purpose`	Multi-step work that needs all tools	Loads CLAUDE.md
`claude`	A catch-all helper	Default for background sessions
statusline-setup, claude-code-guide	Status line setup; questions about Claude Code	Small helpers

A file of your own named Explore replaces the built-in. Turn one off with "permissions": {"deny": ["Agent(Explore)"]}.

Forks: a subagent that inherits the conversation

	FORK	NORMAL SUBAGENT
Context	Your whole conversation	Fresh: only the delegation
Tools, model	Same as your session	From its file
Prompt cache	Shared, so cheaper	Its own
Start it	/subtask <task>, or Claude	Claude delegates, or you name it

Use a fork when

- A fresh agent would need too much background
- You want to try an idea without filling your window

Fork mode is on by default. It keeps output isolation: only the result comes back.

Turn it off with CLAUDE_CODE_FORK_SUBAGENT=0 in the env block of settings. Stop forks with "Agent(fork)" in permissions.deny.

A delegation must be complete

A subagent has no memory of your plan. Each delegation answers four questions.

PART	QUESTION	WEAK	STRONG
Goal	The one thing to achieve?	"Look at the export code"	"Add GET /todos/export.csv"
Inputs	What does it need?	"the file we discussed"	"docs/PRD.md section 4; backend/app/main.py"
Output	What comes back, what shape?	"tell me what you did"	"Files changed, test result, under 150 words"
Stop	When done? What not to do?	(nothing)	"Do not touch frontend/. Stop when tests pass"

A delegation, and the rules that follow

DELEGATION

```
Use the backend subagent.
Goal: add GET /todos/export.csv.
Input: docs/PRD.md section 4,
  backend/app/main.py.
Output: files changed and the
  uv run pytest -q result, under 150 words.
Stop: do not touch frontend/.
```

- Pass paths, not "the file we discussed"
- Paste anything produced earlier in the session
- One job per delegation: "review and fix" is two agents
- Say what the result will be used for
- Name the agent, or @-mention it: @agent-backend



This is the M00 prompt pattern again: role, context, task, output format, constraints.

Foreground or background

	FOREGROUND	BACKGROUND
Your conversation	Waits until it finishes	Keeps going
Permission prompts	Appear in your session	Appear in your session, labelled with its name
Result	In the same turn	Arrives later as a notification
Best for	A step you need before you continue	Independent or long work

Subagents Claude starts run in the background by default. Ctrl+B sends a waiting one to the background. /tasks shows running work.

Claude waits for a result only with fork mode off (CLAUDE_CODE_FORK_SUBAGENT=0). CLAUDE_CODE_DISABLE_BACKGROUND_TASKS=1 makes it always wait.

Rule, skill, subagent or hook?

Files hold knowledge

CLAUDE.md, rules: "Dates are UTC"

Skills hold procedures

/add-todo-field (M6)

Subagents hold isolation

Own context, own tools: todo-reviewer

Hooks hold guarantees

Must happen every time (M9)

They combine: a subagent preloads a skill with its skills field; a skill runs in a fork with context: fork (M6); a subagent carries hooks.

Ways to arrange several subagents (pipelines, reviewers, routers) are in M7B.

7.8 · YOUR TURN

Subagent or not?

For each, choose: stay in the main conversation, a subagent, a skill, or a rule in CLAUDE.md.

1. "Run the whole backend and frontend test suites and tell me only what failed."
2. "Rename the button label from Add to Save."
3. "Get a review of my CSV export from something that did not write it."
4. "Titles are 1 to 200 characters."

REFERENCE

A read-only reviewer: todo-reviewer

.CLAUDE/AGENTS/TODO-REVIEWER.MD

```
---
name: todo-reviewer
description: Reviews the branch diff against the to-do rules R1-R8 and for missing tests,
  and returns a verdict. Use after implementation, before a pull request.
tools: Read, Grep, Glob, Bash
disallowedTools: Write, Edit, NotebookEdit
model: sonnet
maxTurns: 15
---
You never change files. Use Bash only for git diff, git log, git status and the tests.
Review git diff main...HEAD against the rules in CLAUDE.md (R1-R8): UTC dates, title 1-200,
archive not delete, priority and category values, overdue boundary, no clock in rules.py,
no titles in logs. Check that every new endpoint has a test in backend/tests/.
Return exactly:
VERDICT: APPROVE or CHANGES
FINDINGS
1. [high|medium|low] path:line - what is wrong - what to do   ("none" if approving)
Approve only when there are no high or medium findings.
```

REFERENCE

A cheap searcher and a role agent

A CHEAP SEARCHER: .CLAUDE/AGENTS/FINDER.MD

```
---
name: finder
description: Finds where something is
  defined or used and lists file and
  line. Use for "where is X" questions.
tools: Read, Grep, Glob
model: haiku
maxTurns: 8
---
Search backend/ and frontend/src/.
Return a table: file, line, one-line
note. Never edit. Under 150 words.
```

A ROLE AGENT: .CLAUDE/AGENTS/FRONTEND.MD

```
---
name: frontend
description: Implements React UI changes
  under frontend/. Use for components,
  styling and client-side work.
tools: Read, Grep, Glob, Edit, Write, Bash
model: sonnet
---
Edit only files under frontend/.
Write .jsx, never TypeScript. Follow the
components in frontend/src/components/.
Run npm run build before you finish.
Return: files changed, build result.
```

REFERENCE

All fields, limits and removed tools

FIELD	PURPOSE
`permissionMode`	default, acceptEdits, auto, dontAsk, plan ... (M8)
`mcpServers`	MCP servers for this agent only (M10)
`hooks`	Hooks while this agent runs (M9)
`memory`	user, project or local: its own memory folder
`isolation`	worktree: its own temporary worktree (M5)
`effort`	low ... max
`omitClaudeMd`	true: skip CLAUDE.md
`autoCompactWindow`	Compact earlier
`color`	Label colour in the UI
`initialPrompt`	First turn when run with --agent

Limits

- 3 levels of nesting (CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH)
- 20 running at once (CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS)

Always removed from subagents

AskUserQuestion, plan-mode tools, ScheduleWakeup, Workflow. A subagent cannot ask you a question.

REFERENCE

Commands and settings

COMMAND	DOES
`/tasks`	Running subagents; open a transcript
`/subtask <task>`	Start a fork
`@agent-<name> <task>`	Force a specific subagent
`claude --agent <name>`	Run the whole session as it
`claude --agents '<json>'`	A session-only agent
`claude --debug`	Why a file was skipped
`Ctrl+B`	Send a waiting task to background

SETTING OR VARIABLE	EFFECT
`CLAUDE_CODE_SUBAGENT_MODEL`	Model for agents that set none
`CLAUDE_CODE_FORK_SUBAGENT=0`	Fork mode off; Claude can wait
`CLAUDE_CODE_DISABLE_BACKGROUND_TASKS=1`	Always wait
`CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH`	Nesting depth (default 3)
`CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS`	Parallel cap (default 20)
`deny: ["Agent(Explore)"]`	Turn one agent off
`"agent": "<name>"`	Default agent for a project

CHECK YOUR UNDERSTANDING

Question 1

Your todo-reviewer answers a question about "the bug we just discussed" with a generic reply. Why, and what do you change?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 2

You want a reviewer that cannot change code. Which is enforced: "do not edit files" in its prompt, or leaving Edit and Write out of tools? Is that complete if it has Bash?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 3

Two reviewers run in parallel and edit nothing. Two implementers run in parallel and both edit backend/app/main.py. Which is safe, and why?

Think first, then we discuss.

CHECK YOUR UNDERSTANDING

Question 4

You ask Explore to check the code against your rules in CLAUDE.md, and it ignores them. Why?

Think first, then we discuss.